helpful. To incorporate human feedback with AutoGen, one can set human_input_mode='ALWAYS' in the user proxy agent. We select one challenging problem that none of these systems can solve autonomously across three trials. We adhere to the process outlined below to provide human inputs for all the compared methods:

1. Input the problem: Find the equation of the plane which bisects the angle between the planes $3x-6y+2z+5=0$ and $4x-12y+3z-3=0$, and which contains the point $(-5,-1,-5)$. Enter your answer in the form
$$Ax+By+Cz+D=0,$$
where $A$, $B$, $C$, $D$ are integers such that $A > 0$ and $\gcd(|A|,|B|,|C|,|D|)=1$.
2. The response from the system does not solve the problem correctly. We then give a hint to the model: Your idea is not correct. Let's solve this together. Suppose $P=(x,y,z)$ is a point that lies on a plane that bisects the angle, the distance from P to the two planes is the same. Please set up this equation first.
3. We expect the system to give the correct distance equation. Since the equation involves an absolute sign that is hard to solve, we would give the next hint: Consider the two cases to remove the abs sign and get two possible solutions.
4. If the system returns the two possible solutions and doesn't continue to the next step, we give the last hint: Use point (-5,-1,-5) to determine which is correct and give the final answer.
5. Final answer is $\boxed{\text{11x+6y+5z+86=0}}$.

We observed that AutoGen consistently solved the problem across all three trials. ChatGPT+Code Interpreter and ChatGPT+Plugin managed to solve the problem in two out of three trials, while AutoGPT failed to solve it in all three attempts. In its unsuccessful attempt, ChatGPT+Code Interpreter failed to adhere to human hints. In its failed trial, ChatGPT+Plugin produced an almost correct solution but had a sign discrepancy in the final answer. AutoGPT was unable to yield a correct solution in any of the trials. In one trial, it derived an incorrect distance equation. In the other two trials, the final answer was incorrect due to code execution errors.

**Scenario 3: Multi-User Problem Solving.** Next-generation LLM applications may necessitate the involvement of multiple real users for collectively solving a problem with the assistance of LLMs. We showcase how AutoGen can be leveraged to effortlessly construct such a system. Specifically, building upon scenario 2 mentioned above, we aim to devise a simple system involving two human users: a student and an expert. In this setup, the student interacts with an LLM assistant to address some problems, and the LLM automatically resorts to the expert when necessary.

The overall workflow is as follows: The student chats with the LLM-based assistant agent through a student proxy agent to solve problems. When the assistant cannot solve the problem satisfactorily, or the solution does not match the expectation of the student, it would automatically hold the conversation and call the pre-defined ask_for_expert function via the *function_call* feature of GPT in order to resort to the expert. Specifically, it would automatically produce the initial message for the ask_for_expert function, which could be the statement of the problem or the request to verify the solution to a problem, and the expert is supposed to respond to this message with the help of the expert assistant. After the conversation between the expert and the expert's assistant, the final message would be sent back to the student assistant as the response to the initial message. Then, the student assistant would resume the conversation with the student using the response from the expert for a better solution. A detailed visualization is shown in Figure 6.

With AutoGen, constructing the student/expert proxy agent and the assistant agents is straightforward by reusing the built-in UserProxyAgent and AssistantAgent through appropriate configurations. The only development required involves writing several lines of code for the ask_for_expert function, which then becomes part of the configuration for the assistant. Additionally, it's easy to extend such a system to include more than one expert, with a specific ask_for_expert function for each, or to include multiple student users with a shared expert for consultation.

### A2: Retrieval-Augmented Code Generation and Question Answering

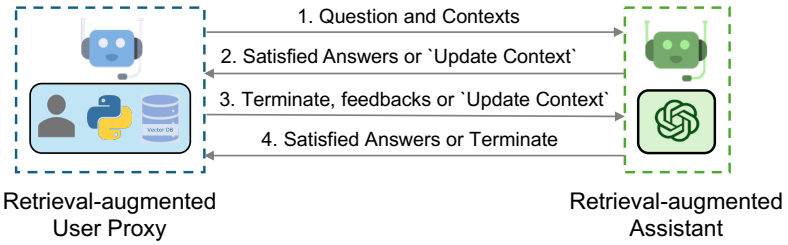


Figure 7: Overview of Retrieval-augmented Chat which involves two agents, including a Retrieval-augmented User Proxy and a Retrieval-augmented Assistant. Given a set of documents, the Retrieval-augmented User Proxy first automatically processes documents—splits, chunks, and stores them in a vector database. Then for a given user input, it retrieves relevant chunks as context and sends it to the Retrieval-augmented Assistant, which uses LLM to generate code or text to answer questions. Agents converse until they find a satisfactory answer.

**Detailed Workflow.** The workflow of Retrieval-Augmented Chat is illustrated in Figure 7. To use Retrieval-augmented Chat, one needs to initialize two agents including Retrieval-augmented User Proxy and Retrieval-augmented Assistant. Initializing the Retrieval-Augmented User Proxy necessitates specifying a path to the document collection. Subsequently, the Retrieval-Augmented User Proxy can download the documents, segment them into chunks of a specific size, compute embeddings, and store them in a vector database. Once a chat is initiated, the agents collaboratively engage in code generation or question-answering adhering to the procedures outlined below:

1. The Retrieval-Augmented User Proxy retrieves document chunks based on the embedding similarity, and sends them along with the question to the Retrieval-Augmented Assistant.
2. The Retrieval-Augmented Assistant employs an LLM to generate code or text as answers based on the question and context provided. If the LLM is unable to produce a satisfactory response, it is instructed to reply with "Update Context" to the Retrieval-Augmented User Proxy.
3. If a response includes code blocks, the Retrieval-Augmented User Proxy executes the code and sends the output as feedback. If there are no code blocks or instructions to update the context, it terminates the conversation. Otherwise, it updates the context and forwards the question along with the new context to the Retrieval-Augmented Assistant. Note that if human input solicitation is enabled, individuals can proactively send any feedback, including Update Context", to the Retrieval-Augmented Assistant.
4. If the Retrieval-Augmented Assistant receives "Update Context", it requests the next most similar chunks of documents as new context from the Retrieval-Augmented User Proxy. Otherwise, it generates new code or text based on the feedback and chat history. If the LLM fails to generate an answer, it replies with "Update Context" again. This process can be repeated several times. The conversation terminates if no more documents are available for the context.

We utilize Retrieval-Augmented Chat in two scenarios. The first scenario aids in generating code based on a given codebase. While LLMs possess strong coding abilities, they are unable to utilize packages or APIs that are not included in their training data, e.g., private codebases, or have trouble using trained ones that are frequently updated post-training. Hence, Retrieval-Augmented Code Generation is considered to be highly valuable. The second scenario involves question-answering on the Natural Questions dataset (Kwiatkowski et al., 2019), enabling us to obtain comparative evaluation metrics for the performance of our system.

**Scenario 1: Evaluation on Natural Questions QA dataset.** In this case, we evaluate the Retrieval-Augmented Chat's end-to-end question-answering performance using the Natural Questions dataset (Kwiatkowski et al., 2019). We collected 5,332 non-redundant context documents and 6,775 queries from HuggingFace. First, we create a document collection based on the entire context corpus and store it in the vector database. Then, we utilize Retrieval-Augmented Chat to answer the questions. An example (Figure 8) from the NQ dataset showcases the advantages of the *interactive retrieval* feature: *"who carried the usa flag in opening ceremony"*. When attempting to answer this question, the context with the highest similarity to the question embedding does not contain the

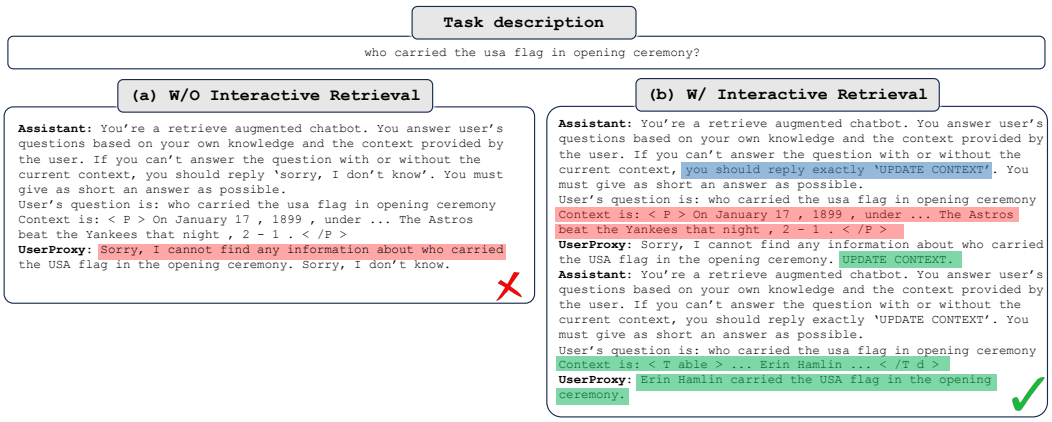


Figure 8: Retrieval-augmented Chat without (W/O) and with (W/) *interactive retrieval*.

required information for a response. As a result, the LLM assistant (GPT-3.5-turbo) replies *"Sorry, I cannot find any information about who carried the USA flag in the opening ceremony. UPDATE CONTEXT."* With the unique and innovative ability to update context in Retrieval-Augmented Chat, the user proxy agent automatically updates the context and forwards it to the assistant agent again. Following this process, the agent is able to generate the correct answer to the question.

In addition, we conduct an experiment using the same prompt as illustrated in (Adlakha et al., 2023) to investigate the advantages of *AutoGen W/O interactive retrieval*. The F1 score and Recall for the first 500 questions are 23.40% and 62.60%, respectively, aligning closely with the results reported in Figure 4b. Consequently, we assert that *AutoGen W/O interactive retrieval* outperforms *DPR* due to differences in the retrievers employed. Specifically, we utilize a straightforward vector search retriever with the *all-MiniLM-L6-v2* model for embeddings.

Furthermore, we analyze the number of LLM calls in experiments involving both *AutoGen* and *AutoGen W/O interactive retrieval*, revealing that approximately 19.4% of questions in the Natural Questions dataset trigger an "Update Context" operation, resulting in additional LLM calls.

**Scenario 2: Code Generation Leveraging Latest APIs from the Codebase.** In this case, the question is *"How can I use FLAML to perform a classification task and use Spark for parallel training? Train for 30 seconds and force cancel jobs if the time limit is reached."*. FLAML (v1) (Wang et al., 2021) is an open-source Python library designed for efficient AutoML and tuning. It was open-sourced in December 2020, and is included in the training data of GPT-4. However, the question necessitates the use of Spark-related APIs, which were added in December 2022 and are not encompassed in the GPT-4 training data. Consequently, the original GPT-4 model is unable to generate the correct code, due to its lack of knowledge regarding Spark-related APIs. Instead, it erroneously creates a non-existent parameter, $spark$, and sets it to True'. Nevertheless, with Retrieval-Augmented Chat, we provide the latest reference documents as context. Then, GPT-4 generates the correct code blocks by setting $use\_spark$ and $force\_cancel$ to True'.

### A3: Decision Making in Text World Environments

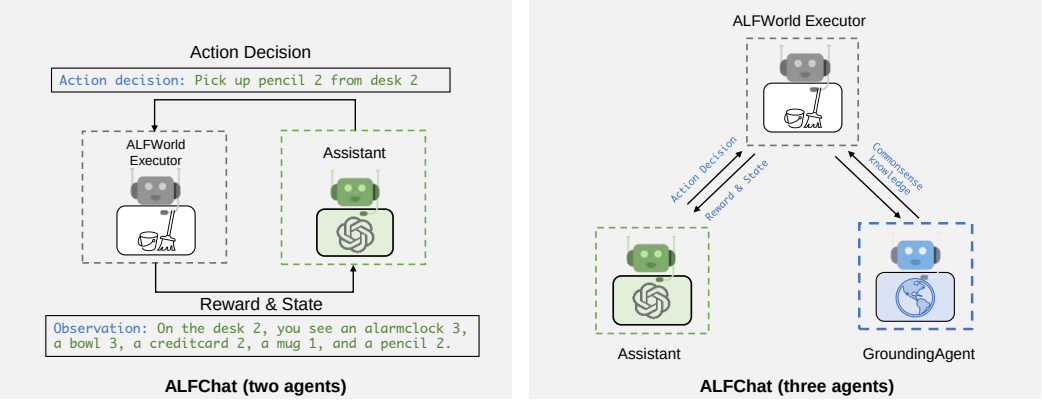


Figure 9: We use `AutoGen` to solve tasks in the ALFWorld benchmark, which contains household tasks described in natural language. We propose two designs: a two-agent design where the assistant agent suggests the next step, and the Executor executes actions and provides feedback. The three-agent design adds a grounding agent that supplies commonsense facts to the executor when needed.

ALFWorld (Shridhar et al., 2021) is a synthetic language-based interactive decision-making task. It comprises textual environments that aim to simulate real-world household scenes. Given a high-level goal (e.g., putting a hot apple in the fridge) and the description of the household environment, the agent needs to explore and interact with the simulated household environment through a textual interface. A typical task environment contains various types of locations and could require more than 40 steps to finish, which highlights the need for agents to decompose the goal into subtasks and tackle them one by one, while effectively exploring the environments.

**Detailed Workflow.** We first propose a straightforward two-agent system with `AutoGen`, illustrated on the left-hand side of Figure 9, to tackle tasks from this benchmark. The system consists of an assistant agent and an executor agent. The assistant agent generates plans and makes action decisions to solve the tasks. The executor agent is tailored specifically for ALFWorld. It performs actions proposed by the assistant and reports action execution results in the household environment as feedback to the assistant. Due to the strict format requirements for the output format, we use the BLEU metric to evaluate the similarity of the output to all valid action options. The option with the highest similarity will be chosen as the action for this round.

One major challenge encompassed in ALFWorld is commonsense reasoning. The agent needs to extract patterns from the few-shot examples provided and combine them with the agent's general knowledge of household environments to fully understand task rules. More often than not, the assistant tends to neglect some basic knowledge of the household environment. Thanks to the easy-to-implement multi-agent conversational feature of `AutoGen`, enhancing the assistant agent's reasoning ability by adding a new grounding agent to provide commonsense facts for the decision-making agent's reference becomes straightforward. By scrutinizing the failed attempts and summarizing the reasons for failure, we obtained a holistic understanding of the commonsense knowledge that the assistant agent lacks. Then, we set a grounding agent to provide this general knowledge when the task begins and whenever the assistant outputs the same action three times in a row. This ensures the assistant takes this commonsense knowledge into consideration and prevents it from getting stuck in outputting the same content or constantly apologizing.

We compare our system's performance with ReAct, which treats ALFWorld as a text-completion task. ReAct (Yao et al., 2022) is a few-shot prompting technique that interleaves reasoning and acting, allowing for greater synergy between the two and significantly improving performance on both language and decision-making tasks. We integrate ReAct into `AutoGen` by modifying the prompts in a conversational manner. Following ReAct, we employ a two-shot setting. The few-shot prompts are obtained from the corresponding repository. As shown in Table 3, the two-agent

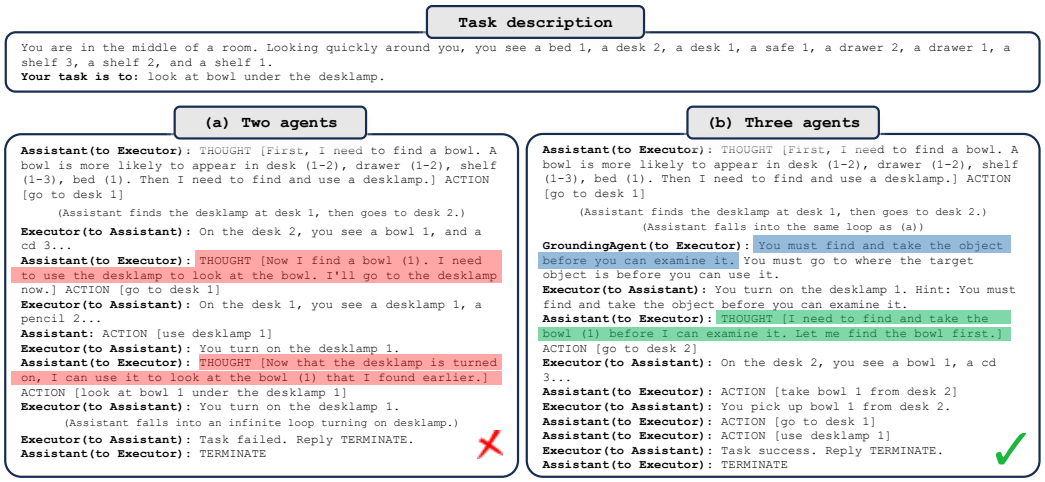


Figure 10: Comparison of results from two designs: (a) Two-agent design which consists of an assistant and an executor, (b) Three-agent design which adds a grounding agent that serves as a knowledge source. For simplicity, we omit the in-context examples and part of the exploration trajectory, and only show parts contributing to the failure/success of the attempt.

| Method | Pick | Clean | Heat | Cool | Look | Pick 2 | All |
|---|---|---|---|---|---|---|---|
| ReAct (avg) | 63 | 52 | 48 | 71 | 61 | 24 | 54 |
| ALFChat (2 agents)(avg) | 61 | 58 | 57 | 67 | 50 | 19 | 54 |
| ALFChat (3 agents)(avg) | 79 | 64 | 70 | 76 | 78 | 41 | 69 |
| ReAct (best of 3) | 75 | 62 | 61 | 81 | 78 | 35 | 66 |
| ALFChat (2 agents)(best of 3) | 71 | 61 | 65 | 76 | 67 | 35 | 63 |
| AFLChat (3 agents)(best of 3) | 92 | 74 | 78 | 86 | 83 | 41 | 77 |

Table 3: Comparisons between ReAct and the two variants of ALFChat on the ALFWorld benchmark. For each task, we report the success rate out of 3 attempts. Success rate denotes the number of tasks successfully completed by the agent divided by the total number of tasks. The results show that adding a grounding agent significantly improves the task success rate in ALFChat.

design matches the performance of ReAct, while the three-agent design significantly outperforms ReAct. We surmise that the performance discrepancy is caused by the inherent difference between dialogue-completion and text-completion tasks. On the other hand, introducing a grounding agent as a knowledge source remarkably advances performance on all types of tasks.

**Case study**. Figure 10 exemplifies how a three-agent design eliminates one root cause for failure cases. Most of the tasks involve taking an object and then performing a specific action with it (e.g., finding a vase and placing it on a cupboard). Without a grounding agent, the assistant frequently conflates finding an object with taking it, as illustrated in Figure 10a). This leads to most of the failure cases in 'pick' and 'look' type tasks. With the introduction of a grounding agent, the assistant can break out of this loop and successfully complete the task

**Takeaways.** We introduced a grounding agent to serve as an external commonsense knowledge source, which significantly enhanced the assistant's ability to make informed decisions. This proves that providing necessary commonsense facts to the decision-making agent can assist it in making more informed decisions, thus effectively boosting the task success rate. `AutoGen` brings both simplicity and modularity when adding the grounding agent.